

CST 205 : Object Oriented Programming using Java

Series 2 Answer Key

PART A

1. Write an example of implementing an interface in Java.

```
interface Animal {  
    public void animalSound(); // no body  
    public void sleep(); // no body  
}  
  
class A1 implements Animal {  
    public void animalSound() { // body  
        System.out.println("A1 says: wee wee");  
    }  
    public void sleep() { //body  
        System.out.println("Zzz ");  
    }  
}  
  
class Inter{  
    public static void main (String[] args) {  
        A1 a1=new A1();  
        a1.animalSound( );  
        a1.sleep( );  
    }  
}
```

2. Explain any three Byte Stream classes in Java.

1. ObjectInputStream

- The ObjectInputStream class extends the InputStream class and implements the ObjectInput interface. ObjectInputStream is responsible for reading objects from a stream. A constructor of this class is

ObjectInputStream(InputStream inStream) throws IOException

where inStream is the input stream from which serialized objects should be read.

Important Methods

- Object readObject() - Reads an object from the invoking stream
- void close() - Closes the invoking stream.

2. ObjectOutputStream

- The `ObjectOutputStream` class extends the `OutputStream` class and implements the `ObjectOutput` interface. It is responsible for writing objects to a stream. A constructor of this class is

`ObjectOutputStream(OutputStream outStream)` throws `IOException`

where `outStream` is the output stream to which serialized objects will be written.

Important Methods

- `final void writeObject(Object obj)`- Writes `obj` to the invoking stream
- `void close( )` -Closes the invoking stream
- `void flush( )` -flushes the output buffers

3. `FileInputStream`

- `FileInputStream` class is an input stream to read data from a file in the form of sequence of bytes

To read data from a file,

1. Create `FileInputStream` class object and pass filename as the parameter to the constructor

```
FileInputStream fileobject;
```

```
fileobject = new FileInputStream("Sample.txt");
```

2. Use a version of `read( )` that is defined within `FileInputStream`.

```
int read( ) throws IOException
```

```
eg. int c=fileobj.read();
```

- Each time `read()` called, it reads a single byte from the file and returns the byte as an integer value.

3. Differentiate between checked and unchecked exceptions in Java with examples.

Checked Exception

- The classes which directly inherit `Throwable` class except `RuntimeException` and `Error` are known as checked exceptions
- e.g. `IOException`, `SQLException` etc.
- Checked exceptions are checked at compile-time.

Unchecked Exception

- The classes which inherit `RuntimeException` are known as unchecked exceptions
- e.g. `ArithmeticException`, `NullPointerException`,
- Unchecked exceptions are not checked at compile-time, but they are checked at runtime

4 Explain any three methods of Thread class.

- public void run() - used to perform action for a thread.
- public void start() -starts the execution of the thread. JVM calls the run() method on the thread.
- public void sleep(long milliseconds) -Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds.
- public void join() -waits for a thread to die.
- public int getPriority() -returns the priority of the thread.
- public int setPriority(int priority) - changes the priority of the thread.
- public String getName()-returns the name of the thread.
- public Thread currentThread()-returns the reference of currently executing thread.
- public void stop() -used to stop the thread(deprecated).

5 Illustrate the working of any two methods of String class that compare strings.

equals()

- To compare two strings for equality, use equals()
- General form:

boolean equals(Object str)

- Here, String object str is compared with the invoking String object. It returns true if the strings contain the same characters in the same order, and false otherwise. The comparison is case-sensitive.

equalsIgnoreCase()

- This performs a comparison that **ignores case differences**(not case sensitive)
- When it compares two strings, it considers A-Z to be the same as a-z.
- General form:

boolean **equalsIgnoreCase(String str)**

Eg:

String s1="javatpoint";

String s2="javatpoint";

String s3="JAVATPOINT";

String s4="python";

System.out.println(s1.equals(s2));**//true** because content and case is //same

```
System.out.println(s1.equals(s3));//false because case is not same  
System.out.println(s1.equals(s4));//false because content is not same
```

PART B

6 Write a java program to show the significance of method overriding in achieving run time polymorphism. Discuss difference between method overriding and method overloading.

```
class Vehicle{  
    //defining a method  
    void run(){System.out.println("Vehicle is running");}  
}  
//Creating a child class  
class Bike2 extends Vehicle{  
    //defining the same method as in the parent class  
    void run(){System.out.println("Bike is running safely");}  
  
    public static void main(String args[]){  
        Bike2 obj = new Bike2();//creating object  
        obj.run();//calling method  
    } }  

```

Method Overloading	Method Overriding
Used to increase the readability of the program.	Used to provide the specific implementation of the method that is already provided by its super class.
Performed within class.	Occurs in two classes that have IS-A (inheritance) relationship
Parameter must be different.	Parameter must be same
Example of compile time polymorphism.	Example of run time polymorphism.
Cannot be performed by changing return type of the method only	Return type must be same or covariant in method overriding.

7. Develop a Java package named 'evenpackage', with a class Even containing a static method that check whether a number is even or not, and returns that information. Import this package in another class and use to check a number is even or not.

```
//create a folder evenpackage and save the Even.java in this folder  
  
package evenpackage;
```

```

public class Even {
    public static boolean isEven(int number)
    {
        if(number % 2 == 0)
            return true;
        else
            return false;
    }
}

```

// Create MainClass.java file outside the evenpackage folder

```

import evenpackage.Even;
import java.util.Scanner;
public class MainClass {
    public static void main(String[] args) {
        Scanner sc =new Scanner(System.in);
        System.out.println("Enter a number");
        int n=sc.nextInt();
        // Using the isEven method from the Even class in the evenpackage
        boolean f=Even.isEven(n);
        if (f) {
            System.out.println(n + " is even.");
        } else {
            System.out.println(n + " is odd.");
        }
    }
}

```

Steps to compile and run package

```
javac evenpackage/Even.java
```

```
javac MainClass.java
```

```
java MainClass
```

8.Explain the concept of Serialization and demonstrate how an object can be serialized with a sample program.

- Serialization is the process of writing(converting) the state of an object to a byte stream.This is useful when we want to save(store) the state of the program to a persistent(permanent) storage area, such as a file or when we want to send it over network.Later we can restore these objects by using the process of deserialization.Deserialization converts byte streams into object.

Eg: of Java Serialization

- We are going to serialize the object of Student class. The writeObject() method of ObjectOutputStream class provides the functionality to serialize the object. We are saving the state of the object in the file named f1.txt.

```
import java.io.*;

class Student implements Serializable{
    int r;
    String name;
    public static void main(String args[]){
        try {
            Student s1=new Student(); // Creating the object
            s1.r=12;
            s1.name="Ravi";
            //Creating stream and writing the object
            FileOutputStream fout= new FileOutputStream("f1.txt");
            ObjectOutputStream out=new ObjectOutputStream(fout);
            out.writeObject(s1);
            out. flush ( ) ;
            out.close(); // closing the stream
            System.out.println( "success " ) ; }
            catch(Exception e)
            {
                System.out.println(e);
            }
        }
    }
}
```

9.Write a Java program to copy the contents of one file to another file using FileInputStream and FileOutputStream class

```
import java.io.*;

class Filecopy
```

```

{
public static void main(String args[]) throws IOException
{
    FileInputStream f1=null;
    FileOutputStream f2=null;
    try
    {
        f1=new FileInputStream("test.txt");
        f2= new FileOutputStream("cp.txt");
        int c;
        do
        {
            c=f1 .read();
            if(c!=-1)
            {
                f2.write((char)c);
                System.out.print((char)c);
            }
        }while(c!=-1);
    }
    catch(FileNotFoundException e){
        System.out.println("Filenotfound");
        return;
    }
    f1 .close();
    f2.close(); }}

```

10 Demonstrate the significance of the keywords 'try', 'catch', 'finally', 'throw' and 'throws' in exception handling of Java with appropriate examples.

- Java exception handling is managed via five keywords:
 - **try** : Program statements that we want to monitor for exceptions are contained within a **try** block.
 - **throw** : To manually throw an exception.
 - **throws** : Any exception that is thrown out of a method must be specified by a **throws** clause.
 - **catch** : to catch exceptions and handle it in some rational manner.
 - **finally** : Any code that absolutely must be executed before a method returns is put in a **finally** block.

//explain each keyword

Exception Handling

- General Form :

ExceptionType is the type of exception that has occurred.

```
try
{
    // block of code to monitor for errors
}
catch (ExceptionType1 exOb)
{
    // exception handler for ExceptionType1
}
catch (ExceptionType2 exOb)
{
    // exception handler for ExceptionType2
}
// ...
finally
{
    // block of code to be executed before try block ends
}
```

```
import java.util.Scanner;
```

```
class AgeVerify
```

```
{
```

```
void vote(int age) throws IllegalAccessException
```

```
{
```

```
try
```

```
{
```

```
if (age < 18)
```

```
{
```

```
throw new IllegalAccessException("You must be at least 18 years old to vote.");
```

```
}
```



```

else
{
System.out.println("You can vote!");
}
}
catch(Exception e)
{
System.out.println("EXCEPTION OCCURED: "+e);
}
finally
{
System.out.println("Finally age verification done.");
}
}
}
public class Excep
{
public static void main(String[] args) throws IllegalAccessException
{
Scanner sc=new Scanner(System.in);
int age;
System.out.println("Enter age of voter.");
age=sc.nextInt();
AgeVerify av=new AgeVerify();
av.vote(age);
System.out.println("Verification completed successfully");
}
}

```

11 a) Discuss the methods of creating threads in Java

Java defines two ways for creating a thread :

1.implementing the Runnable interface.

- The Runnable interface should be implemented by any class whose instances are intended to be executed by a thread.

- Runnable interface have only one method named run().

public void run() - used to perform action for a thread.

Steps to create a new Thread using Runnable

- Create a Runnable implementer and implement run() method.
- Instantiate Thread class and pass the implementer to the Thread, Thread has a constructor which accepts Runnable instance.
- Invoke start() of Thread instance, start internally calls run() of the implementer. Invoking start(), creates a new Thread which executes the code written in run().

```
class Multi3 implements Runnable{
public void run(){
System.out.println("thread is running...");
}
public static void main(String args[]){
Multi3 m1=new Multi3(); //object of implementer created
Thread t1 =new Thread(m1); // Using the constructor Thread(Runnable r)
t1.start();
} }
```

2. Extending the Thread class

- The Thread class provide constructors and methods to create and perform thread operations
- The Thread class extends Object class and implements Runnable interface
- Commonly used Constructors of Thread class:
 - Thread()
 - Thread(String name)
 - Thread(Runnable r)
 - Thread(Runnable r, String name)

```
class MyThread extends Thread{
public void run(){
System.out.println("Current thread name:"+ Thread.currentThread().getName());
System.out.println ("run method called");
}}
class ThreadClass {
public static void main(String[] args){
```

```
MyThread t = new MyThread();  
t.start();// create a new thread and it execute run()  
}}
```

b) Write a Java program that creates multiple child threads to print odd and even numbers from 50-100

```
class EvenThread extends Thread {  
    public void run() {  
        for (int i = 50; i <= 100; i += 2)  
        {  
            System.out.println(" EvenThread "+i );  
            try{  
                sleep(1000);  
            }  
            catch (InterruptedException e)  
            {  
                e.printStackTrace();  
            } } }  
}
```

```
class OddThread extends Thread {  
    public void run() {  
        for (int i = 51; i <= 100; i += 2)  
        {  
            System.out.println(" OddThread "+i );  
            try  
            {  
                sleep(1000);  
            }  
            catch (InterruptedException e)  
            {  
                e.printStackTrace();  
            } } }  
}
```

```

public class Oddeven50{
    public static void main(String[] args) {
        EvenThread evenThread = new EvenThread();
        evenThread.start();
        OddThread oddThread = new OddThread();
        oddThread.start();
    }
}

```

12 What are the uses of synchronized keyword in Java? Explain with examples.

- When we start two or more threads within a program, there may be a situation when multiple threads try to access the same resource and finally they can produce unforeseen result due to concurrency issues.
- Synchronization in java is the capability of controlling the access of multiple threads to any shared resource.
- For example, if multiple threads try to write within a same file then they may corrupt the data because one of the threads can override data or while one thread is opening the same file at the same time another thread might be closing the same file.
- So there is a need to synchronize the action of multiple threads and make sure that only one thread can access the resource at a given point in time.

a)using synchronized method

- With a synchronized method, the lock is obtained for the duration of the entire method.
- So if you want to lock the whole object, use a synchronized method

synchronized void methodname ()

eg: implementation of synchronized method

```

class First{
    synchronized public void display(String msg) {
        System.out.print "["+msg);
        try {
            Thread.sleep(500);
        }
        catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

```

```

System.out.println("]");
}}
class Second extends Thread {
String msg;
First fobj;
Second (First fp,String str) {
fobj = fp;
msg= str;
start();
}
public void run() {
fobj.display(msg);
}}
public class MyThreadDemo
{
public static void main (String[] args)
{
First fnew = new First();
Second ss = new Second(fnew,"welcome");
Second ss1= new Second(fnew,"new");
Second ss2= new Second(fnew, "programmer");
}}

```

b)Using Synchronized block

- If we want to synchronize access to an object of a class or only a part of a method to be synchronized then we can use synchronized block for it.
- It is capable to make any part of the object and method synchronized.
- With synchronized blocks we can specify exactly when the lock is needed. If you want to keep other parts of the object accessible to other threads, use synchronized block.
- General form of the synchronized statement :

```

synchronized(object identifier)
{ // Access shared variables and other shared resources
}

```

Eg:

```
class First{
public void display(String msg) {
System.out.print "["+msg);
try {
Thread.sleep(1000);
}
catch(InterruptedException e) {
e.printStackTrace();
}
System.out.println("]");
}}

class Second extends Thread {
String msg;
First fobj;
Second (First fp,String str) {
fobj = fp;
msg= str;
start();
}

public void run() {
synchronized(fobj)//synchronized block
{
    fobj.display(msg);
} }}

public class SyncBlock
{
public static void main (String[] args)
{
First fnew = new First();
Second ss = new Second(fnew,"welcome");
Second ss1= new Second(fnew,"new");
```

```
Second ss2= new Second(fnew, "programmer");
```

```
}}
```